

Implementation Reference Map

Uncertainty-Conditioned RL with Evidential Policy Networks

Antonio Galdes

Last updated: March 23, 2026

1 Evidential Deep Learning Architecture

- **NIG parameterisation** ($\gamma, \nu, \alpha, \beta$) with softplus constraints: `evidential_policy.py`, `sb3_integration.py`. Based on ?.
- **Evidential regularisation** $L_R = |\mathbf{a} - \gamma| \cdot (2\nu + \alpha)$ with $\lambda_{reg} = 0.01$: `sb3_integration.py:EvidentialPPO.train`, `configs/train_config.yaml`. Coefficient matches EPPO $\xi = 0.01$?, validated range $[0.001, 0.1]$ by ? and ?.
- **Evidential actor (not critic)**: Design decision in `CLAUDE.md`. Note: EPPO ? applies evidential DL to the *critic*, not the actor. Our evidential actor is a less-studied configuration that produces Student-t policy distributions.
- **NIG hyperprior initialisation** (TODO in `train_config.yaml`): ν bias ≈ 0.5 , α bias ≈ 0.5 , β bias ≈ 0.0 . Produces “honestly uncertain” starting points. EPPO ? uses $\text{Gamma}(2,1)$ for ν/α , $\text{Gamma}(1,1)$ for β .
- **Lambda annealing** from 0 to 0.01 over 50K steps (TODO): Lets NLL establish good predictions before regularisation begins. Standard practice in evidential DL literature.

2 PPO Hyperparameters

- **Learning rate 3e-4 with linear decay**: `train_ppo.py:linear_schedule()`, `configs/train_config.yaml`. 3e-4 validated by ? (250K+ agents). Linear decay proven optimal by ?.
- **batch_size=256, n_epochs=5** (40 off-policy steps, down from 320): `configs/train_config.yaml`. ? showed ~ 15 off-policy steps outperformed 959 on CARLA. Corrected upward to 40 for single-env setup (no parallel data collection).
- **ent_coef=0.0** (zero entropy for continuous control): `configs/train_config.yaml`. ?: “no evidence that the entropy term improves performance on continuous control environments.”
- **target_kl=0.02** (early epoch stopping): `train_ppo.py`, `configs/train_config.yaml`. Standard PPO mechanism ?. Set slightly higher than typical 0.015 due to noisy reward landscape from weather randomisation.
- **gamma=0.99** (kept, not reduced to 0.98): Scenario-specific: with current reward structure, success bonus at step ~ 300 needs $\gamma^{300} \approx 0.05$ to propagate. At $\gamma = 0.98$, $\gamma^{300} \approx 0.002$ (invisible). SB3 highway-env parking uses $\gamma = 0.95$? but with dense reward per step.
- **n_steps=2048** (kept for single CARLA env): ~ 4 episodes per update provides trajectory diversity across weather conditions. ? used lower values but with 4–8 parallel envs.

- **Observation normalisation** (VecNormalize, norm_obs=True): `train_ppo.py`. ?: “one of the most impactful implementation details.”
- **norm_reward=False**: `train_ppo.py`. Auto-normalisation divides by $std(discountedreturns)$, unstable when reward distribution shifts (early: penalties only, later: success bonuses). With potential-based shaping, manual scaling preferred.

3 Network Architecture

- **net_arch=[256,256]** for both actor and critic: `configs/train_config.yaml`. Evidential head outputs $4 \times 3 = 12$ NIG parameters from 18-dim state. ? recommend separate sizing; kept equal because the 18-dim state includes 9 covariance features encoding complex non-linear uncertainty structure, plus 3 relative target pose features.
- **ReLU activation**: Both EPPO ? and ? use ReLU.
- **LayerNorm** (TODO): EPPO ? uses LayerNorm after each hidden layer. Preserves network plasticity under non-stationary training ??.

4 Reward Function

- **Potential-based distance shaping** (TODO, Task 9): Switch from $R = -d$ to $R = d_{t-1} - d_t$. Grounded in ? (policy invariance under potential-based transformations).
- **Milestone bonuses** at distance thresholds (TODO): ? found milestone-augmented rewards outperformed both sparse and pure dense rewards for parking with PPO.
- **highway-env parking benchmark** as reference design: ? uses weighted p-norm with $p = 0.5$, weights $[1.0, 0.3, 0.0, 0.0, 0.02, 0.02]$.

5 Environment Design / Curriculum and Bay Sampling

- **Stratified bay type sampling** (1/3 each: perpendicular, angled, parallel): `carla_parking.py:CARLAParking`. `configs/train_config.yaml: bay_type_sampling`. Justified on three grounds:
 - ?: balanced difficulty distribution leads to faster convergence than training on the natural (skewed) distribution. The parking lot has far more perpendicular bays than parallel bays in the wild; without stratification the agent would rarely see the hardest manoeuvre.
 - ?: rare difficult scenarios (parallel pull-in bays) are never practised enough without deliberate curriculum design. The stratified sampler is the minimal curriculum intervention that guarantees each bay type is seen equally often.
 - ? highway-env ParkingEnv uses 28 same-type slots as the direct precedent for a goal-conditioned parking environment. This environment extends the design to 15 bays across 3 types with stratified sampling.
 - ?: general instance diversity principle for RL generalisation. Cited only for the general principle – the specific count thresholds (4000–16000 CoinRun levels) do not transfer to vector-state parking, where the image-based pixel-memorisation failure mode does not apply.

6 Environment Design and Domain Randomisation

- **Randomised floor plan presets (2 training, 1 OOD holdout):** `configs/train_config.yaml:parking`. Two floor plan geometries (`rectangle`, `trapezoid`) are sampled uniformly during training. One geometry (`irregular_a`, five-sided polygon) is held back entirely and used exclusively for OOD evaluation. Justified by domain randomisation [?](#): exposing the policy to geometric variety during training encourages generalisation. The held-out geometry drives epistemic uncertainty higher on novel layouts, demonstrating the evidential policy’s calibration. Why 3 floor plans rather than more: the agent’s relative-obs formulation (`dx`, `dy`, `dyaw`) makes absolute bay position in world coordinates invisible – overfitting to specific bay positions is structurally impossible. The real diversity axes are manoeuvre type (3 bay types) and lot perimeter shape. Two training floor plans provide sufficient geometric variety. See also [?](#) for the general diversity principle (note: their 4000–16000 CoinRun level counts do not transfer to vector-state parking – cited for the general principle only).
- **Randomised bay occupancy per episode** ($\sim 70\%$ probability per eligible slot): Prevents the policy from memorising a fixed obstacle layout. The target slot and parallel-bay neighbours are always kept empty. Also justified by domain randomisation [?](#).
- **Randomised weather, NPC patrol count, pedestrian count per episode:** Varies the aleatoric uncertainty distribution across episodes. 0–3 patrol vehicles (scripted proportional controller, no Traffic Manager), 0–6 pedestrians (`WalkerControl`, zone-constrained random-walk, no NavMesh required). [?](#).
- **Pedestrian zone-boundary correction (steer toward zone centre):** `carla_parking.py:update_pedes`. Pedestrians are assigned to aisle zones defined in the layout YAML. When a walker reaches within 0.5 m of the zone boundary, it is assigned a new heading directed toward the zone centre rather than using velocity reflection. Pure reflection causes oscillation: the walker overshoots by one physics step, exits the zone, the heading is negated, and on the next step the walker is still outside and is negated again, producing a standing vibration on the boundary wall. The “steer toward interior” approach is the same mechanism used by the CARLA Scenario Runner [?](#), which assigns a new interior waypoint when an actor approaches a trigger-region boundary. Pedestrians remain in the aisles between parked car bodies, which is the correct location for LiDAR occlusion effects.
- **Three bay types (perpendicular, angled 45 deg, parallel forward entry):** `configs/layouts/*.yaml`. All three bay types appear in real car parks, including within the same facility [?](#). Training across all three bay types improves generalisation [?](#). Bay type is not in the observation; the agent infers the required manoeuvre from the relative target pose (`dx`, `dy`, `dyaw`) and reward signal.
- **Bay dimensions from German EAR 05 / EU harmonised practice (FGSV 2005):** Perpendicular 2.5 m $\times$ 5.0 m, aisle 6.0 m; angled 2.5 m $\times$ 5.4 m, aisle 3.6 m; parallel 2.5 m $\times$ 8.0 m (forward pull-in without reversing), aisle 4.0 m. EAR 05 represents current comfortable minimum standards, slightly larger than absolute minimums, appropriate for a learning agent. Source: FGSV Empfehlungen für Anlagen des ruhenden Verkehrs (EAR 05), 2005.
- **Reserved slots for parallel forward entry:** Slots immediately in front and behind the target parallel bay are marked `always_empty` in the layout YAML (`configs/layouts/*.yaml:bays[*].always_`
`true`). Engineering decision: no reference required. Perpendicular and angled bays require no reserved neighbours as the ego drives straight in without sweeping adjacent slots.

7 Experimental Design and Evaluation

- **10 seeds minimum** for final ablation: `configs/train_config.yaml:random_seeds`. ? showed 5 seeds produce misleading results. ? showed 5 seeds yields insufficient statistical power. ? established IQM + bootstrap CIs as gold standard.
- **reliable evaluation protocol**: Report IQM with 95% stratified bootstrap CIs, probability of improvement, performance profiles. ?.
- **Sequential seeds [0..9]**: Avoids appearance of cherry-picking. Standard practice per ?.
- **1M total timesteps**: EPPO ? uses 500K. ? used 60M with curriculum learning. 1M provides 2× safety margin for compact state + shaped rewards.

8 System Framing and Sensor Architecture

- **Decoupled perception/control architecture**: Target parking pose (x, y, θ) is provided by an upstream visual perception module (e.g. camera-based bay detector). This system focuses on the control layer only: navigating safely to the given pose under localisation uncertainty. This matches commercial autonomous parking pipelines ?? where perception and control are decoupled. Onboard bay detection is out of scope and noted as future work in the dissertation.
- **GNSS excluded**: GNSS is unusable in covered or indoor parking lots due to building occlusion and multipath interference. Not feasible for real-world deployment. All sensor configurations therefore use dead-reckoning and scan-matching only.
- **Wheel odometry excluded from EKF sensor suite**: The CARLA ROS bridge publishes `/carla/ego_vehicle/odometry` from ground-truth physics state (not integrated wheel encoder ticks). It carries zero noise and zero covariance, so feeding it into the EKF as `odom0` suppresses covariance variation regardless of lot conditions. Wheel odometry is therefore removed from the EKF config; the prediction step uses IMU only between LiDAR scan-matching updates. Justified by:
 - ?: autonomous parking in GPS-denied lots using LiDAR + IMU only (no wheel odometry), explicitly motivated for slow-speed short-duration manoeuvres.
 - ?: IMU + 2D LiDAR EKF (no wheel odometry) for indoor localisation in GNSS-denied environments.
 - ?: real-time EKF (IMU + 2D LiDAR, no wheel odometry) for UGV indoor navigation at centimetre-level accuracy.
 - ?: the `robot_localization` package explicitly supports IMU-only prediction; wheel odometry is optional when a scan-matching odometry source (Cartographer) is available.
 - Parking manoeuvres are short ($\sim 10\text{--}30\text{ s}$) and low-speed ($< 2\text{ m/s}$). IMU-only drift over this duration is bounded and corrected at each LiDAR update. No encoder hardware required for real-world deployment.

Config change: `configs/ros2_config.yaml` – `odom0` entry and `odom0_config` removed.

- **Suite A sensor configuration (primary)**: 2D LiDAR + IMU. 2D LiDAR (SICK TiM / Hokuyo style, CARLA single-channel `sensor.lidar.ray_cast`) at bumper height with Cartographer scan-matching ?. Localises against vertical surfaces: parked car bodies, kerbs, bollards. Low hardware cost (500–1500 GBP), feasible for real-world deployment. IMU feeds the EKF prediction step between LiDAR scans. Covariance variation arises naturally: fewer

parked cars = sparser LiDAR features = higher localisation uncertainty. See ??? for precedent.

- **Suite B sensor configuration (upgrade):** 3D LiDAR + IMU. Upgrade to multi-layer 3D LiDAR (Velodyne VLP-16 style) if Suite A covariance variation is insufficient. Multiple vertical layers detect ground-plane features (bay edges, kerbs) in addition to vertical surfaces. Higher hardware cost (3000–8000 GBP).
- **Suite C sensor configuration (extended):** 3D LiDAR + IMU + RGB camera with visual odometry ? for appearance-based loop closure. Last resort if neither Suite A nor B produces sufficient covariance variation. Highest implementation effort.
- **Why start with 2D LiDAR:** Sim-to-real feasibility. A 2D LiDAR is affordable for a real parking lot deployment. In the parking lot scenario, localisation relies on vertical returns from parked car bodies rather than ground-plane markings, so a single horizontal scan plane at bumper height is sufficient. Fewer parked cars naturally produces sparser features and higher uncertainty – which is the covariance variation signal needed for the degradation study.

9 Infrastructure

- **CARLA simulator:** ?. Headless, vector observations only (no camera rendering).
- **Stable-Baselines3:** ?. PPO implementation, VecNormalize, callbacks, custom policy subclassing.
- **robot localisation EKF:** ?. Provides covariance output as uncertainty signal in observation space.
- **Google Cartographer (algorithm):** ?. 2D SLAM via scan-matching. Provides `/scan_matched_odometry` with covariance for EKF fusion (all sensor suites).
- **Cartographer ROS documentation:** ?. Consulted for ROS integration, topic names, and to confirm `use_imu_data=false` is correct for 2D SLAM. Relevant files: `configs/cartographer/cartographer_configs/cartographer_config_3d.lua`.
- **Cartographer upstream 2D defaults:** ?. Used to verify that parameter overrides in Suite A config are deliberate departures from library defaults (e.g. tighter grid resolution for parking lot scale).
- **Cartographer `revo_lds.lua` example:** ?. Closest published example for a single-channel 2D LiDAR (Suite A). Corroborates `optimize_every_n_nodes=35` and `num_laser_scans=1` choices.
- **Cartographer `backpack_2d.lua` example:** ?. Confirms that `num_point_clouds=1` with a 2D trajectory builder is officially supported. Justifies Suite B/C config feeding 3D LiDAR `PointCloud2` into a 2D Cartographer trajectory builder.
- **CARLA ROS bridge TF issue #750:** ?. Known conflict between CARLA bridge TF output and Cartographer’s TF. Relevant to `ros2/launch/carla_bridge.launch.py` and the `ros2-bridge` Dockerfile. Workaround needed before end-to-end integration testing.